

SPEAR: Code-Augmented Agentic Prompt Optimization

Mengyin Lu, Cong Feng, Huimin Han, Guangming Lu, Yu Sun,
Xiaonan Ding, Shihui Long, Fengyi Li, Tanvi Motwani

LinkedIn Corporation

{melu, cofeng, huhan, glu, ysun1, kding, shlong, fenli, tmotwani}@linkedin.com

Abstract

Automatic prompt engineering (APE) rewrites prompts to improve downstream task performance, but existing APE loops treat the optimizer itself as a fixed pipeline. We port the code-as-action paradigm of CodeAct (Wang et al., 2024a) to APE and propose *SPEAR* (Sandboxed Prompt Engineer with Active Roll-back), a free-form agentic optimizer with four tools – `evaluate`, `python`, `set_prompt`, `finish` – that decides autonomously how and when to use them. The distinctive tool is the Python sandbox: the optimizer writes and executes arbitrary Python on the current evaluation DataFrame, performing structural error analysis (confusion matrices, error clustering, per-group metrics) the agent itself authors. Two guardrails turn the long-horizon agent into a monotone-improving optimizer: *auto-rollback* on metric regression, and an optional *guard-metric floor*. We evaluate on three industrial LLM-as-judge suites (13 judge tasks across recruiter-intake, conversational-memory, and query-refinement systems) plus seven BBH tasks and GSM8K. *SPEAR* wins every industrial task on the primary metric (κ 0.857 vs 0.359 on tool-selection; F1-macro 0.815 vs 0.763 on filter-relevance; κ 0.254 vs 0.218 on the hardest extraction dimension). On BBH-7 *SPEAR* averages 0.938 accuracy vs GEPA 0.628 and TextGrad 0.484. Ablations show the Python tool is the largest single lever on complex judge tasks ($\Delta \approx +0.79\kappa$ on the 5-class tool-selection judge, $\Delta \approx +0.35\kappa$ on the hardest extraction dimension when removed); its irreplaceable contribution is class-pair confusion aggregation that a long-context LLM cannot extract reliably from the raw eval DataFrame.

1 Introduction

Automatic prompt engineering (APE) rewrites a seed prompt to maximise a metric on a labeled dataset. Existing systems (Yang et al., 2024; Guo et al., 2024; Yuksekgonul et al., 2025; Agrawal

et al., 2026) all share one structural trait: the optimizer is a *fixed pipeline*. Its error signal – scalar loss, Pareto front, or single-row textual critique – is chosen at design time, and the analysis the optimizer performs is whatever the pipeline pre-bakes. When errors cluster by structure (a confused class pair, a label-rule contradiction visible only when grouping by a feature), a fixed pipeline cannot expose that structure to the optimizer.

Our proposal. We replace the fixed APE pipeline with a free-form agent equipped with a Python sandbox, transplanting the *code-as-action* paradigm of CodeAct (Wang et al., 2024a) into the APE loop. *SPEAR* has four tools – `evaluate` (score the current prompt and return per-row predictions), `python` (run arbitrary code over that DataFrame), `set_prompt` (rewrite), `finish` – and is the first APE system in which the optimizer *actively authors* analysis code over the evaluation DataFrame (positioning in Appendix T). A concrete payoff: on a 5-class CMA judge whose seed prompt yields $\kappa=0.20$ because it collapses minorities into the majority, *SPEAR*’s first python block builds the EXPECTED $\times$ PARSED confusion matrix, targets the most-confused class pair, and four such rewrites take κ from 0.20 to 0.95+. Two minimal guardrails – metric-based *auto-rollback* on regression and an optional *guard-metric floor* – turn the long-horizon agent into a monotone-improving optimizer.

Contributions. (1) **Code-as-action applied to APE.** The first APE system in which the optimizer actively authors analysis code over the evaluation DataFrame; the closest prior work, OPTO/Trace (Cheng et al., 2024), treats execution traces as *passive* feedback. (2) **Free-form agent vs. fixed pipeline.** An ablation replacing the agent with a rigid same-tools cycle loses 26 pp on BBH `logical_deduction_5obj` and 0.27κ on Hiring Assistant *job location*: orchestration au-

tonomy carries a substantial share of the lift. (3) **Industrial case studies of what code-as-action discovers in APE.** Two studies (§5) expose qualitatively distinct rewrites the affordance enables – a confusion-matrix-driven category-definition rewrite on a multi-class judge, and a label-rule-contradiction discovery on a binary judge.

2 Related work

Existing APE systems – LLM-in-loop search (Zhou et al., 2023; Yang et al., 2024), evolutionary methods (Fernando et al., 2023; Guo et al., 2024; Zehle et al., 2025; Agrawal et al., 2026), surrogate/beam and tree search (Opsahl-Ong et al., 2024; Pryzant et al., 2023; Wang et al., 2024b), symbolic and joint search (Schnabel and Neville, 2024; Agarwal et al., 2024), self-supervised (Xiang et al., 2025), memory-augmented (Liang et al., 2026; Gangireddi et al., 2026), gated refinement/compression (Shi et al., 2025; Xu et al., 2025; Jiang et al., 2024), and textual-gradient methods (Yuksekogonul et al., 2025) – all use a fixed optimizer loop; the optimizer never authors analysis code over per-row predictions. The closest adjacent work, OPTO/Trace (Cheng et al., 2024), treats execution traces as *passive* feedback. A concurrent wave of judge-prompt and multi-agent APE systems (Zhou et al., 2024; Wen et al., 2025; Li et al., 2025; Zhang et al., 2026; Zhou et al., 2026; Zhang et al., 2025; Long et al., 2025; Liu et al., 2026c,b; Rishav et al., 2026; Liu et al., 2026a; Singhal et al., 2026; Breen Herrera et al., 2026) similarly lacks an active code-execution affordance. Among these, Build/Judge/Optimize (Breen Herrera et al., 2026) is closest to our industrial setting – it calibrates an LLM-as-judge for a production multi-agent shopping assistant – but optimizes via natural-language reflection on multi-turn trajectories rather than over-DataFrame code execution. SPEAR ports the *code-as-action* paradigm of CodeAct (Wang et al., 2024a) into APE (lineage: ReAct (Yao et al., 2023), Reflexion (Shinn et al., 2023), ADAS (Hu et al., 2024)); auto-rollback is in the family of GRACE (Shi et al., 2025). We benchmark against TextGrad and GEPA (both open-source, run unchanged); a full positioning table is in Appendix T. We do not benchmark DSPy/MIPRO (Opsahl-Ong et al., 2024) because its search space jointly covers instructions *and* few-shot demonstrations, but our LLM-as-judge prompts cannot accommodate demonstrations (each row is the held-out scoring

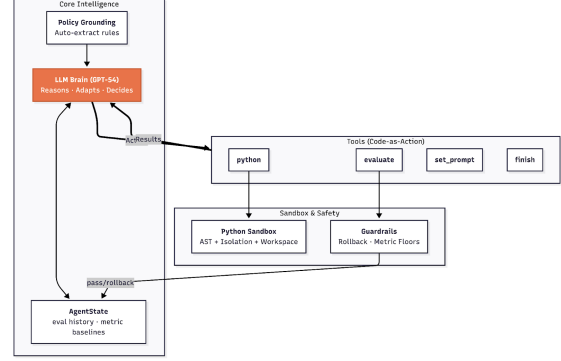


Figure 1: SPEAR architecture. A GPT-5.4 *agent brain* is grounded by an optional policy document and chooses one of four tools per step: python (executes arbitrary code in an AST-restricted sandbox with the current eval DataFrame in scope), evaluate (scores the current prompt on train or valid via a task-model inference pass), set_prompt (replaces the system prompt), and finish (terminates). Tool outputs (Results) flow back to the brain and update an AgentState that tracks eval history and best-metric baselines. Two guardrails sit between tool calls and the state: auto-rollback reverts set_prompt mutations when the primary metric regresses below its best-seen value, and a guard-metric floor rejects rewrites that would sink a user-nominated secondary metric. No component requires a fixed evaluate–analyze–rewrite ordering; the agent decides.

target, so no row can serve as an in-context demo without label leakage), reducing MIPRO’s search to instruction-only – a strict subset of what TextGrad and GEPA already optimize. OPRO and Evo-Prompt BBH numbers are cited from the published papers (Appendix J).

3 Method

3.1 Four tools, no pipeline

The optimizer seeks $P^* = \arg \max_{P \in \mathcal{P}} \frac{1}{N} \sum_i g(\pi(f(P, x_i)), y_i)$ on a labeled dataset $\{(x_i, y_i)\}_{i=1}^N$, where f is the fixed task model, π parses the structured prediction from the generated text, and $g(\hat{y}, y)$ is the primary metric (e.g. Cohen’s κ , macro-F1), subject to a budget of full-dataset evaluations $\#evals \leq B$ and an optional guard-metric floor $g_2 \geq \tau$ (§3.2). The dataset is split into train (analysis), valid (selection), and test (held out); formal expansion in Appendix A. SPEAR’s agent (Figure 1) runs a plain while loop and selects one of four tools per step (full pseudo-code in Appendix B): **evaluate(split, row_indices=None)** runs the prompt on train/valid (optionally a subset) and

returns per-row predictions; one full call consumes a budget unit. **python(code)** executes code in a sandbox with pandas/numpy, the eval DataFrame, the current prompt (read-only), and an llm() helper (default GPT-4o, $T=0.2$). **set_prompt(new)** replaces the system prompt; **finish(summary)** terminates. Defaults: `max_eval_calls=20`, `max_steps=5 × max_eval_calls`; the initial evaluate is forced. Valid is exposed as a *scoring*, not *analysis*, surface: the sandbox sees aggregate label distributions on valid but never row content (redaction details and the brain prompt in Appendices C, N).

3.2 Guardrails

Auto-rollback reverts the prompt when a full-split evaluate drops the primary metric below its best-seen value; sampled evaluations do not update best-seen state, so the agent cannot inflate the headline by selectively evaluating easy rows. **Guard metric** is opt-in: a second metric with a floor τ that rewrites must not sink below (used on Facet with precision, $\tau \approx 0.80$ auto-initialised from the seed prompt; the other 12 dimensions do not need it because κ already penalises majority collapse). Algorithm in Appendix B.

3.3 Sandbox and safety

The python tool runs in an AST-restricted sandbox (module whitelist, no network/shell, 60 s wall-clock + 512 MB RSS cap, read-only prompt) under a cooperative threat model (trusted optimizer LLM, internal data); sandbox details in Appendix C, eval-tool reliability instrumentation in Appendix D.

4 Experiments

4.1 Setup

Tasks. Three industrial LLM-as-judge benchmarks – *Hiring Assistant* (10 extraction-judge dimensions on a recruiter-intake assistant; train=94, valid=130, κ), *CMA* (2 tool-selection judges on a conversational memory agent; 125 rows, 60/40 split, κ), and *Facet Suggestion* (1 filter-relevance judge; 943 rows, 70/30 split, macro-F1) – with adjudicated human labels (Hiring Assistant: three annotators plus expert adjudication; full per-dataset descriptions and labeling protocols in Appendix Q). We also evaluate on *BBH-7* (Suzgun et al., 2023) (seven OPRO-style tasks, 50/50 split, accuracy) and *GSM8K* (Cobbe et al., 2021) (260 train / 1,319 test, accuracy).

Methods, models, budgets. Task model: GPT-4o ($T = 0$, 4k tokens). Optimizer model: GPT-5.4 – an internal deployment of a GPT-5-family reasoning model – for SPEAR and GEPA. TextGrad’s backward engine is GPT-4o (GPT-5.4 produced persistent gRPC errors on long Hiring Assistant backward prompts); the like-for-like comparison repeats the industrial setup with TextGrad+GPT-5.4 in the unified-optimizer view (Appendix G). We release the agent harness, prompts, and sanitized data; the specific GPT-5-family point release is the only un-reproducible component. Baselines: TextGrad (textgrad pip) and GEPA (gepa==0.1.1), run unchanged. Budgets: SPEAR `max_eval_calls=15–20`; GEPA `max_metric_calls=400–500`; TextGrad `max_steps=5` (batch 5–10) – comparable total task-LM calls across methods (§4.4; cost detail in Appendix E).

Seeds and best-of- N . Industrial headline numbers are the best validation metric across N independent optimizer runs ($N \in \{3, 5\}$ for SPEAR, $N \in \{1, 3\}$ for baselines; per-cell N in Appendix I). This optimistic “best-of- N ” framing is the standard APE convention; we bound the run-to-run variance directly (Hiring Assistant location σ up to 0.22) in Appendix H and complement it with a single-seed unified-optimizer view in Appendix G.

4.2 Main results (industrial)

Table 1 reports the primary metric per task. SPEAR wins 11 of 12 directly comparable industrial tasks (excluding the train-only *CMA tool-redundancy* row, which has no baseline; it is reported for completeness only); the sole exception is Hiring Assistant *unsound inference*, a near-ceiling dimension where TextGrad reaches $\kappa=0.968$ vs SPEAR’s 0.937 (all methods >0.88). Largest absolute wins are on the hardest dimensions: Hiring Assistant *job location* (κ 0.76 vs 0.11/0.36), *CMA tool-missing* (κ 0.976 vs 0.69/0.63, driven by the Python-tool confusion-matrix manoeuvre, Appendix F), and Facet (F1-macro 0.846 vs 0.751/0.734). Six cells have $\Delta \geq 0.30$, well outside the seed-variance bound from the multi-seed experiment in Appendix H (per- N counts in Appendix I).

4.3 Public benchmarks (BBH, GSM8K)

SPEAR averages 0.938 accuracy across BBH-7 vs GEPA 0.628 and TextGrad 0.484 (following

Task	Metric	SPEAR	TextGrad	GEPA
Hiring Assistant: unsound inference	κ	0.937	0.968	0.889
Hiring Assistant: job location	κ	0.760	0.110	0.361
Hiring Assistant: employment type	κ	0.779	0.444	0.494
Hiring Assistant: workplace type	κ	0.852	0.355	0.716
Hiring Assistant: organization	κ	0.881	0.685	0.881
Hiring Assistant: location format	κ	0.961	0.614	0.000
Hiring Assistant: required skills	κ	0.938	−0.020	0.168
Hiring Assistant: over-qualification	κ	0.915	0.002	0.205
Hiring Assistant: multi-qualification merge	κ	0.635	0.279	0.198
Hiring Assistant: per-qualification unsound	κ	0.849	0.000	0.000
CMA: tool-missing	κ	0.976	0.694	0.631
CMA: tool-redundancy	κ	1.000	—	—
Facet Suggestion	F1-macro	0.846	0.751	0.734 [†]

Table 1: Industrial best-prompt performance, best across N independent optimizer runs (best-of- N). SPEAR runs $N \in \{3, 5\}$, baselines $N \in \{1, 3\}$ in the canonical configuration; total N_{total} per cell is asymmetric – on three Hiring Assistant dimensions (unsound, location, employment) SPEAR has up to $N_{\text{total}} = 28$ runs vs. baselines’ 2, so deltas on those cells should be read jointly with the single-seed unified-optimizer view (Table 2); per-cell counts in Appendix I. TextGrad’s reflection engine is GPT-4o (GPT-5.4 produced gRPC errors on long Hiring Assistant backward prompts); SPEAR and GEPA use GPT-5.4. [†]GEPA’s short-budget Facet run returned the seed unchanged; 0.734 is from the longer-budget run. *CMA tool-redundancy* is train-only and not directly comparable to the other 12 rows; included for completeness.

OPRO convention, BBH uses the optimization valid split as the reported test set; see §4). A large share of the BBH-7 gap is driven by *output-format rewriting* rather than architectural lift: on the three format-diverging tasks (date_understanding, disambiguation_qa, logical_deduction_5obj) the seed regex cannot produce the target surface form and SPEAR rewrites the contract on the first evaluate; on the four non-format tasks the mean gap shrinks to +0.033 over GEPA, with overlapping CIs on 3 of 4. On GSM8K (matched format) no method moves the metric (SPEAR 0.965, GEPA 0.960, TextGrad 0.956). The setting where SPEAR’s architecture is load-bearing is the industrial judge-task setting (§4.2). Full per-task table with bootstrap CIs in Appendix R.

4.4 Ablations, transfer, and cost

Components (Appendix S). Six single-seed ablations on a five-task subset isolate the contribution of each SPEAR component. The Python tool (A1) is the single largest lever: removing it costs -0.35κ on Hiring Assistant location and -0.79κ on CMA tool-missing (CI non-overlap with SPEAR full); these gaps measure the value of *arbitrary analysis* on top of the same textual error summary TextGrad/GEPA already consume. An aggregation-vs-information probe (A6: no Python tool, but the full train eval DataFrame is

appended to every evaluate result as text) shows the Python tool’s contribution is task-structured rather than just “richer info”: A6 closes 100% of the A1-to-full gap on logical_deduction_5obj (format-rewriting), 72% on Hiring Assistant location (single-pattern rule discovery), 39% on Facet, but only 2% on CMA tool-missing (5-class confusion-matrix structure), and is net-negative on BBH causal_judgement (long context distracts a small-lift task). The Python tool’s distinctive contribution is specifically the ability to aggregate per-row predictions into class-pair confusion structure that a long-context LLM does not extract reliably from raw text. A free-form agent vs. a same-tools rigid loop (A4) loses 0.27κ on Hiring Assistant location and 26 pp on logical_deduction_5obj: orchestration autonomy is load-bearing. Downgrading the agent model from GPT-5.4 to GPT-4o (A5) causes near-total failure (e.g., 0.000 on logical_deduction_5obj): model quality is a threshold condition. Auto-rollback (A2) and forced init eval (A3) sit within single-seed CI of SPEAR full and we do not claim them as significant mean-improvers; they raise the optimizer’s *floor* (no SPEAR run terminates below seed across 78 logged runs).

Unified-optimizer view (Appendix G). With all baselines upgraded to GPT-5.4, SPEAR still wins every industrial task on the primary metric: CMA $\kappa=0.857$ vs. TextGrad 0.359; Facet F1=0.815 vs.

TextGrad 0.763; Hiring Assistant location $\kappa=0.254$ vs. GEPA 0.218 (within seed band, directional).

Robustness, transfer, cost (Appendices H, K, E). A held-out re-split confirms SPEAR has the highest test κ on 2 of 3 Hiring Assistant dimensions (exception: *employment_type* where GEPA test $\kappa=0.691 > \text{SPEAR } 0.377$ under high variance). A 3-seed run on the hardest dimension gives SPEAR mean $\kappa=0.315$ ($\sigma=0.160$) vs. GEPA 0.149 vs. TextGrad 0.105 – SPEAR’s worst seed equals GEPA’s best. Cross-model transfer: SPEAR has positive retention on 6/6 task $\times$ target-model pairs vs. 4/6 for GEPA and TextGrad. Cost: SPEAR converges in 2–3 full evals (vs. GEPA’s ~ 500), and total USD-per-task is within 17% of GEPA across the three industrial datasets (USD figures include sandbox 11m() calls; per-component breakdown in Appendix E).

5 Case study: what code-as-action discovers

Hiring Assistant job location – label-contradiction discovery (Appendix P). The judge’s seed prompt yields $\kappa=0.035$ (IAA ceiling 0.74); SPEAR reaches 0.76. On step 3, the agent’s python tool clusters failing rows by the agent-emitted location string and finds that commute-based phrases (“within commutable distance to ...”) dominate. A second python block cross-references commute phrases against whether the recruiter actually stated a base city and discovers a *label-rule contradiction*: rows where the agent emitted geographically incoherent commute constraints (“commutable distance to US/EMEA”) were nonetheless labeled *pass*. The resulting rewrite – a rule change, not a reasoning scaffold – whitelists recruiter-stated locations as sole evidence and adds an explicit “invalid commute constraint” failure path. This is the paper’s core qualitative claim: code execution lets the optimizer *discover* rules, not only refine them.

CMA tool-missing – multi-class confusion analysis (Appendix F). On a 5-class judge whose seed prompt yields $\kappa=0.20$ because it collapses minorities into the majority, SPEAR’s first python block builds the EXPECTED $\times$ PARSED confusion matrix, targets the most-confused class pair, and four such rewrites take κ from 0.20 to 0.95+. Baselines’ text-critique-driven rewrites stall at $\kappa=0.63/0.69$ because they cannot see which class pairs are confused.

6 Discussion

When does the Python tool matter? It dominates when errors cluster by feature (Hiring Assistant, CMA); on near-saturated tasks (BBH object_counting, GSM8K) it adds little. The affordance is uniquely useful for complex judge tasks and complementary on others.

Failure modes. Across 78 archived SPEAR runs, 71 (91%) materially improved (best $> \text{init} + 0.01$), 7 (9%) plateaued, and *none* ended strictly below seed – the empirical signature of auto-rollback. Within the plateau bucket we observe three recurring modes: plateau-and-stop, metric-flipping rewrite (the failure the guard-metric floor catches on Facet), and parse-failure inflation.

Format vs. reasoning. The BBH-7 lead is driven primarily by first-eval output-format rewriting on format-diverging tasks; on GSM8K (matched format) no method moves the metric. The industrial judge tasks are where SPEAR’s architectural advantage is load-bearing.

7 Conclusion

SPEAR applies the code-as-action pattern (Wang et al., 2024a) to automatic prompt engineering: a free-form agent writes and executes Python over the evaluation DataFrame to analyze its own errors, paired with metric-based auto-rollback and an optional guard-metric floor. On twelve industrial judge-prompt tasks with full three-method comparison (plus one SPEAR-only train-only task), seven BBH tasks, and GSM8K, SPEAR matches or beats TextGrad and GEPA on the primary metric in the like-for-like comparison, with the largest absolute wins on complex multi-class judge tasks. Ablations show the Python tool is the single largest contributor on those tasks, with its irreplaceable role – not recovered by appending the raw eval DataFrame to the agent’s context – being class-pair confusion aggregation on multi-class judges; the BBH-7 lead is driven by output-format rewriting and does not survive on GSM8K. Model quality is a necessary condition; guardrails raise the optimizer’s floor rather than its mean.

Limitations

Reproducibility of the optimizer model. GPT-5.4 refers to an internal deployment of a GPT-5-family reasoning model accessed via our organization’s gateway; we cannot publicly disclose

which point release maps to this alias. External researchers should expect to reproduce our results with the strongest publicly available GPT-5-family reasoning model at the time of replication; we will release the agent harness, prompts, and sanitized data so that the only un-reproducible component is the specific point release.

Proprietary data. BBH and GSM8K are public; Hiring Assistant, CMA, and Facet rest on internal annotation, and the optimized prompts on those tasks cannot be released due to proprietary policy content (BBH/GSM8K prompts will be released). We release a synthetic judge-task stub and the single-seed unified-optimizer numbers (Appendix G).

Statistical reach. (i) GPT-4o at $T = 0$ is not bit-reproducible on borderline rows (σ up to 0.28 across replicas). (ii) Single-seed ablations: deltas ≤ 0.05 should be read as directional; only the multi-seed experiment (Appendix H) bounds across-seed variance. (iii) Small- n judges: CMA *tool-missing* valid is ~ 50 rows / 5 classes; we cross-check with the unified-optimizer view (Appendix G, $\Delta\kappa = 0.498$).

Read-side valid exposure. SPEAR sees aggregate label-distribution on valid (not row content – redacted, see §3); this is asymmetric vs. GEPA/TextGrad. The held-out test and multi-seed analysis (Appendix H) – where SPEAR’s worst seed equals GEPA’s best – bound any residual adaptation. We cannot distinguish between read-side exposure and small- n dev-set artifact (baselines also exhibit positive dev→test gaps) with current data; both may contribute.

Prompt length and annotation bias. SPEAR’s optimized prompts grow $\sim 2\times$ in length, comparable to GEPA/TextGrad rewrites; production may pair with a post-hoc prompt compressor (e.g., LongLLMLingua (Jiang et al., 2024), ProCut (Xu et al., 2025)) or a length-penalising APE such as CAPO (Zehle et al., 2025). Judge prompts aligning with human-adjudicated labels inherit the annotation pipeline’s biases.

Sandbox is not adversarial. The threat model assumes a trusted optimizer LLM and internal data; AST-level whitelisting is known-leaky, so a deployment with user-supplied DataFrame content would need stronger isolation (container or WASM). Across 200+ logged runs we observed no

escape attempts; this is operational experience, not a security evaluation.

Ethical Considerations

Data and consent. The three industrial benchmarks rest on internal annotation pipelines for a recruiter-facing extraction assistant, a conversational-memory agent, and a job-search query-refinement system. Annotations are produced by paid in-house annotators under our organization’s annotation guidelines; no end-user content is released. The sanitized data and synthetic judge-task stubs we release contain no PII and no proprietary policy text.

Bias and fairness. The recruiter-intake judge scores extraction over free-text input; downstream uses can affect hiring workflows. Aligning a judge to adjudicated labels inherits any annotation-pipeline bias and does not, on its own, address representational fairness. Production deployments should pair SPEAR-optimized judges with bias monitoring on protected attributes and periodic re-adjudication.

Misuse and safety. SPEAR’s Python sandbox is for offline optimization on labeled internal data, not for serving end-user traffic. The threat model is co-operative (trusted optimizer LLM, internal data); a user-facing deployment with untrusted DataFrame content would require stronger isolation (container or WASM, §3.3).

Acknowledgments

We thank our teammates at LinkedIn for foundational engineering and infrastructure support across the LLM-serving gateway, the evaluation harness, and the production judge deployments underlying every experiment in this paper. We are also indebted to the annotation team and the in-house linguistic experts whose adjudicated labels constitute the gold standard for all three industrial benchmarks; this work would not exist without their careful and patient labelling. The authors declare no competing financial interests outside the submitted work.

References

Eshaan Agarwal, Joykirat Singh, Vivek Dani, Raghav Magazine, Tanuja Ganu, and Akshay Nambi. 2024. PromptWizard: Task-aware prompt optimization framework. *arXiv preprint arXiv:2405.18369*.

- Lakshya A. Agrawal, Shangyin Tan, Dilara Soylu, Noah Ziemis, Rishi Khare, Krista Opsahl-Ong, Arnav Singhvi, Herumb Shandilya, Michael J. Ryan, Meng Jiang, Christopher Potts, Koushik Sen, Alexandros G. Dimakis, Ion Stoica, Dan Klein, Matei Zaharia, and Omar Khattab. 2026. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations (ICLR)*. Oral; open-source release: `gepa==0.1.1`.
- Alejandro Breen Herrera, Aayush Sheth, Steven G. Xu, Zhucheng Zhan, Charles Wright, Marcus Yearwood, Hongtai Wei, Sudeep Das, Danny Nightingale, Meg Watson, and Charles Pollnow V. 2026. Build, judge, optimize: A blueprint for continuous improvement of multi-agent consumer assistants. *arXiv preprint arXiv:2603.03565*. Calibrated LLM-as-judge pipeline aligned with human annotations; MAMuT GEPA jointly optimizes prompts across agents with trajectory-level scoring; domain: production grocery assistants.
- Ching-An Cheng, Allen Nie, and Adith Swaminathan. 2024. Trace is the next AutoDiff: Generative optimization with rich feedback, execution traces, and LLMs. *arXiv preprint arXiv:2406.16218*.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, Lukasz Kaiser, Matthias Plappert, Jerry Tworek, Jacob Hilton, Reiichiro Nakano, Christopher Hesse, and John Schulman. 2021. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*.
- Chrisantha Fernando, Dylan Banarse, Henryk Michalewski, Simon Osindero, and Tim Rocktäschel. 2023. Promptbreeder: Self-referential self-improvement via prompt evolution. *arXiv preprint arXiv:2309.16797*.
- Balaji Dinesh Gangireddi, Aniketh Garikaparthi, Manasi Patwardhan, and Arman Cohan. 2026. REVERE: Reflective evolving research engineer for scientific workflows. In *ICLR 2026 Workshop on Recursive Self-Improvement*. Reflective optimization that performs targeted edits across system prompt, task-prompt template, and a cumulative cheatsheet; domain: research-coding agents.
- Qingyan Guo, Rui Wang, Junliang Guo, Bei Li, Kaitao Song, Xu Tan, Guoqing Liu, Jiang Bian, and Yujiu Yang. 2024. Connecting large language models with evolutionary algorithms yields powerful prompt optimizers. In *International Conference on Learning Representations (ICLR)*.
- Shengran Hu, Cong Lu, and Jeff Clune. 2024. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*.
- Huiqiang Jiang, Qianhui Wu, Xufang Luo, Dongsheng Li, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. 2024. LongLLMLingua: Accelerating and enhancing LLMs in long context scenarios via prompt compression. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 1658–1677, Bangkok, Thailand. Association for Computational Linguistics.
- Jiajie Li, Huayi Zhang, Peng Lin, Jinjun Xiong, and Wei Xu. 2025. Auto-prompt ensemble for LLM judge. *arXiv preprint arXiv:2510.06538*. Discovers evaluation dimensions from failures and ensembles judge prompts via a Collective Confidence mechanism.
- Guanbao Liang, Yuan Chen Bei, Sheng Zhou, Yuheng Qin, Huan Zhou, Bingxin Jia, Bin Li, and Jiajun Bu. 2026. Generalizable self-evolving memory for automatic prompt optimization. *arXiv preprint arXiv:2603.21520*. MemAPO: dual-memory (successful strategies / error patterns) framework that accumulates cross-task knowledge for generalizable prompt optimization.
- Shiyan Liu, Qifeng Xia, Qiyun Xia, Yisheng Liu, Xinyu Yu, and Rui Qu. 2026a. Reflection in the dark: Exposing and escaping the black box in reflective prompt optimization. *arXiv preprint arXiv:2603.18388*. VISTA: multi-agent APO that decouples hypothesis generation from prompt rewriting, with parallel verification.
- Shu Liu, Shubham Agarwal, Monishwaran Maheswaran, Mert Cemri, Zhifei Li, Qiuyang Mang, Ashwin Naren, Ethan Boneh, Audrey Cheng, Melissa Z. Pan, Alexander Du, Kurt Keutzer, Alvin Cheung, Alexandros G. Dimakis, Koushik Sen, Matei Zaharia, and Ion Stoica. 2026b. EvoX: Meta-evolution for automated discovery. *arXiv preprint arXiv:2602.23413*. Jointly evolves candidate solutions and the search strategy that generates them; benchmarks against AlphaEvolve, OpenEvolve, GEPA, ShinkaEvolve.
- Xingyan Liu, Xiyue Luo, Linyu Li, Ganghong Huang, Jianfeng Liu, and Honglin Qiao. 2026c. SkillForge: Forging domain-specific, self-evolving agent skills in cloud technical support. In *Proceedings of the 49th International ACM SIGIR Conference on Research and Development in Information Retrieval (Industry Track)*. Three-stage pipeline (diagnose failures, pinpoint skill deficiencies, rewrite skills); domain: cloud technical support agents.
- Do Xuan Long, Xingchen Wan, Hootan Nakhost, Chen-Yu Lee, Tomas Pfister, and Serkan Ö. Arik. 2025. VISTA: A test-time self-improving video generation agent. *arXiv preprint arXiv:2510.15831*. Multi-agent iterative prompt optimization for video generation; tournament-based selection plus multi-dimensional (visual / audio / contextual) critique.
- Krista Opsahl-Ong, Arnav Singhvi, Omar Khattab, Kyle Shrivastava, Siddharth Heidari, Matei Zaharia, and Christopher Potts. 2024. Optimizing instructions and demonstrations for multi-stage language model programs. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- Reid Pryzant, Dan Iter, Jerry Li, Yin Tat Lee, Chengguang Zhu, and Michael Zeng. 2023. Automatic

- prompt optimization with “gradient descent” and beam search. In *Empirical Methods in Natural Language Processing (EMNLP)*.
- Rishav Rishav, Pushpak Pujari, and Pushpendre Rastogi. 2026. ContraPrompt: Contrastive prompt optimization via dyadic reasoning trace analysis. *arXiv preprint arXiv:2604.17937*. Contrastive analysis of success/failure CoT trace pairs; reports gains over GEPA on HotPotQA, GDPR-Bench, GPQA Diamond, and BBH.
- Tobias Schnabel and Jennifer Neville. 2024. Symbolic prompt program search: A structure-aware approach to efficient compile-time prompt optimization. *arXiv preprint arXiv:2404.02319*.
- Wenhao Shi, Yiren Chen, Shuqing Bian, Xinyi Zhang, Kai Tang, Pengfei Hu, Zhe Zhao, Wei Lu, and Xiaoyong Du. 2025. No loss, no gain: Gated refinement and adaptive compression for prompt optimization. *arXiv preprint arXiv:2509.23387*. GRACE: gated refinement plus adaptive prompt compression; the gated-refinement mechanism is in the family of update-rejection mechanisms.
- Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. 2023. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Rahul Singhal, Pradyumna Tambwekar, and Karime Maamari. 2026. PrefPO: Pairwise preference prompt optimization. *arXiv preprint arXiv:2603.19311*. LLM-discriminator pairwise-preference APE; matches or exceeds GEPA / MIPRO / TextGrad on 6/9 BBH tasks.
- Mirac Suzgun, Nathan Scales, Nathanael Schärli, Sebastian Gehrmann, Yi Tay, Hyung Won Chung, Aakanksha Chowdhery, Quoc V. Le, Ed H. Chi, Denny Zhou, and Jason Wei. 2023. Challenging BIG-Bench tasks and whether chain-of-thought can solve them. In *Findings of the Association for Computational Linguistics (ACL Findings)*.
- Xingyao Wang, Yangyi Chen, Lifan Yuan, Yizhe Zhang, Yunzhu Li, Hao Peng, and Heng Ji. 2024a. Executable code actions elicit better LLM agents. In *International Conference on Machine Learning (ICML)*.
- Xinyuan Wang, Chi Li, Zhihong Wang, Yiwei Bai, Haozhe Luo, Zhendong Zhang, Zachary Lipton, and Caiming Xiong. 2024b. PromptAgent: Strategic planning with language models enables expert-level prompt optimization. In *International Conference on Learning Representations (ICLR)*. ArXiv:2310.16427.
- Bosi Wen, Pei Ke, Yufei Sun, Cunxiang Wang, Xiaotao Gu, Jinfeng Zhou, Jie Tang, Hongning Wang, and Minlie Huang. 2025. HPSS: Heuristic prompting strategy search for LLM evaluators. In *Findings of the Association for Computational Linguistics (ACL Findings)*. Genetic-algorithm-style search over 8 prompting-strategy factors for LLM evaluators.
- Jinyu Xiang, Jiayi Zhang, Zhaoyang Yu, Xinbing Liang, Fengwei Teng, Jinhao Tu, Fashen Ren, Xiangru Tang, Sirui Hong, Chenglin Wu, and Yuyu Luo. 2025. Self-supervised prompt optimization. *arXiv preprint arXiv:2502.06855*. Reference-free APE via pairwise output comparison judged by an LLM evaluator; reduces sample cost vs. existing methods.
- Zhentao Xu, Fengyi Li, Albert C. Chen, and Xiaofeng Wang. 2025. ProCut: LLM prompt compression via attribution estimation. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pages 285–309, Suzhou (China). Association for Computational Linguistics.
- Chengrun Yang, Xuezhi Wang, Yifeng Lu, Hanxiao Liu, Quoc V. Le, Denny Zhou, and Xinyun Chen. 2024. Large language models as optimizers. In *International Conference on Learning Representations (ICLR)*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*.
- Mert Yuksekogonul, Federico Bianchi, Joseph Boen, Sheng Liu, Pan Lu, Zhi Huang, Carlos Guestrin, and James Zou. 2025. [Optimizing generative ai by backpropagating language model feedback](#). *Nature*, 639(8055):609–616. System name: TextGrad; preprint arXiv:2406.07496.
- Tom Zehle, Moritz Schlager, Timo Heiß, and Matthias Feurer. 2025. CAPO: Cost-aware prompt optimization. *arXiv preprint arXiv:2504.16005*. Evolutionary APE with LLMs as operators; racing for evaluation savings and multi-objective optimization over performance and prompt length.
- Jian Zhang, Zhangqi Wang, Haiping Zhu, Kangda Cheng, Kai He, Bo Li, Qika Lin, Jun Liu, and Erik Cambria. 2026. MARS: Multi-agent adaptive reasoning with socratic guidance for automated prompt optimization. In *AAAI Conference on Artificial Intelligence (AAAI)*. Five-agent (Planner + Teacher-Critic-Student + Target) prompt optimization framed as a POMDP.
- Xinyu Zhang, Yuanquan Hu, Fangchao Liu, and Zhicheng Dou. 2025. P3: Prompts promote prompting. In *Findings of the Association for Computational Linguistics (ACL Findings)*. Jointly optimizes system and user prompts.
- Han Zhou, Xingchen Wan, Yinong Liu, Nigel Collier, Ivan Vulić, and Anna Korhonen. 2024. Fairer preferences elicit improved human-aligned large language model judgments. In *Empirical Methods in Natural Language Processing (EMNLP)*. Introduces ZEPO, a zero-shot evaluator-prompt optimization framework.

Han Zhou, Xingchen Wan, Ruoxi Sun, Hamid Palangi, Shariq Iqbal, Ivan Vulić, Anna Korhonen, and Serkan Ö. Arik. 2026. Multi-agent design: Optimizing agents with better prompts and topologies. In *International Conference on Learning Representations (ICLR)*. MASS: three-stage local-prompt $\rightarrow$ workflow-topology $\rightarrow$ global-prompt search for multi-agent systems.

Yongchao Zhou, Andrei Ioan Muresanu, Ziwen Han, Keiran Paster, Silviu Pitis, Harris Chan, and Jimmy Ba. 2023. Large language models are human-level prompt engineers. In *International Conference on Learning Representations (ICLR)*.

Appendix

A Problem definition (formal)

A prompt $P \in \mathcal{P}$ is system-message text drawn from a free-form prompt space. A fixed task model f maps (P, x) to a generated text $f(P, x)$; a parser π extracts a structured prediction $\hat{y} = \pi(f(P, x))$. The prompt is evaluated on a labeled dataset $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^N$ via metric $g(\hat{y}, y)$ (e.g. Cohen’s κ , macro-F1):

$$P^* = \arg \max_{P \in \mathcal{P}} \frac{1}{N} \sum_{i=1}^N g(\pi(f(P, x_i)), y_i),$$

subject to $\#\text{evals} \leq B$ and, when configured, a guard-metric floor $\frac{1}{N} \sum_i g_2(\pi(f(P, x_i)), y_i) \geq \tau$. Two properties motivate SPEAR’s design: (i) the objective is a black box (prior APE uses reflective LLM proposals or evolutionary search to navigate $\mathcal{P}$); (ii) the per-row terms are individually inspectable, but most existing optimizers reduce them to a scalar aggregate or a one-row textual critique *before* they reach the LLM proposer – SPEAR removes that reduction.

B SPEAR loop (pseudo-code)

Algorithm 1 expands the free-form four-tool loop described in §3, showing the explicit rollback branches for the primary metric and the optional guard-metric floor.

C Sandbox details

The cooperative-mode checks in §3.3 expand to: (i) AST-level module whitelist (pandas, numpy, json, re, collections, itertools, math, statistics); imports outside the whitelist raise. (ii) Network and shell egress stripped: socket, urllib, requests, subprocess, os.system, eval, exec, and file read/write outside two

allow-listed surfaces – a read-only view of the eval DataFrame and a per-run scratch directory `workspace_dir` (used by the brain prompt for cross-step note persistence). (iii) 60 s wall-clock per python call, 512 MB RSS; process isolation is advisory. (iv) the current prompt is exposed as a read-only string; mutations go only through `set_prompt`.

The threat model: a trusted optimizer LLM acting on labelled but otherwise trusted internal data. The sandbox is not adversarial-grade. Data exfiltration risk is bounded by the network-blocking layer; for sensitive data we recommend running on a network-isolated node.

D Eval-tool instrumentation

Three practical issues were load-bearing for reliability of evaluate:

- **Per-row timeouts.** An Azure content-filter or a single slow row can otherwise block the entire evaluation. `evaluate` enforces a per-row wall-clock cap; timed-out rows are recorded as parse failures, which count as wrong under our metric.
- **Strict success predicate.** `eval-lib`’s default `success_predicate` only checked that the provider returned an error-free response, not that the response had non-empty content; a content-filter refusal could silently be counted as a row with empty content and parsed as incorrect. We patch this locally (and upstream) to require non-empty content.
- **Rate-limit recovery.** Cross-model transfer evaluations sometimes accumulated 5–30% of rows into the endpoint’s rate-limit window; naive retry windows (1.5 s) could not recover. We wrap `evaluate` in a post-hoc recovery pass that retries failed rows at 1 qps with 30 s inter-pass sleep.

E Per-method convergence trajectories

Figure 2 plots the per-method optimization curves on the three industrial tasks, supporting the “ ~ 2 –3 full evals to convergence” claim in §4.4 and showing the gap between each baseline’s optimization-internal score and its native-metric end state.

F CMA tool-missing case study

`CMA tool-missing` is a 5-class judge that scores whether a conversational memory agent

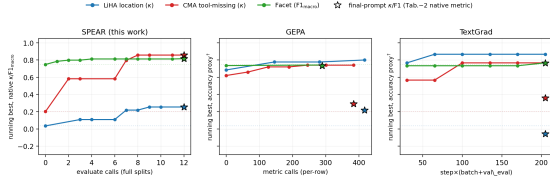


Figure 2: Per-method convergence on the three industrial tasks. Each method is plotted in its own panel because the optimization- internal score and budget unit differ. Per panel: **x-axis** is the method’s native budget unit (SPEAR: full- split evaluate calls; GEPA: metric_calls, i.e. per-row scores within the Pareto loop; TextGrad: step×(batch + val_eval)). **Y-axis** is the metric the optimizer’s loop actually observes: SPEAR logs the native task metric ($\kappa/F1_{\text{macro}}$); GEPA and TextGrad log internal accuracy proxies (DefaultAdapter O/1 accuracy and StringBasedFunction accuracy respectively). The black-edged $\star$ on each curve marks the final-prompt evaluation under the native metric for the same single-seed run (matching Table 2); the gap between the line endpoint and the $\star$ on the GEPA and TextGrad panels (e.g., TextGrad Hiring Assistant-location: line ≈ 0.87 accuracy proxy vs. $\star = -0.057$ native κ) shows that high optimization-internal scores do not translate to high native metric. Dotted horizontals are the seed baseline per task. SPEAR converges in 2–3 full evals on its native metric; GEPA and TextGrad accuracy-proxy plateaus do not match the native- metric $\star$.

invoked the correct retrieval tools for a user query, with labels score|reason-category such as “1|missing_tool_no_retrieval”, “2|partial_missing_tool”, and “3|no_missing_tools_complete”. The seed prompt yields $\kappa=0.202$; SPEAR reaches 0.976 in 28 steps (8 rewrites, 9 evals).

CMA errors are not label-rule contradictions – the labels are consistent. The failure mode is *class confusion*: the seed prompt collapses minority classes (“2|partial_missing_tool”, “1|missing_tool_wrong_type”) into the majority “1|missing_tool_no_retrieval” because its rubric does not distinguish them clearly.

The agent opens with a python block that builds a 5×5 confusion matrix grouped by EXPECTED_OUTPUT $\times$ PARSED_OUTPUT (steps 1, 4), identifies which pairs are confused, reads a few exemplar rows per confused pair, and then rewrites the *category definitions* section of the judge prompt with explicit disambiguating criteria (step 6: κ 0.22 $\rightarrow$ 0.34). The same pattern repeats for the next three rewrites (steps 9, 12, 15), each time targeting the single largest remaining confused pair revealed

by a fresh groupby over the eval DataFrame: κ climbs 0.34 $\rightarrow$ 0.75 $\rightarrow$ 0.85 $\rightarrow$ 0.95. The final two rewrites (steps 19, 22) close the last edge cases to reach 0.976. (This run is the best of five independent SPEAR seeds on CMA tool_missing, in which seed κ ranged 0.86–0.98.)

GEPA’s best rewrite on the same task is a natural-language critique of individual failing examples ($\kappa=0.631$); TextGrad’s textual gradient produces a “reason more carefully about tool selection” scaffold ($\kappa=0.694$). Neither sees *which specific class pairs* are confused because the Pareto candidate selection (GEPA) and the backward-pass (TextGrad) operate over per-example scalar losses, not a typed prediction DataFrame. Access to the DataFrame is a hard requirement for the confusion-matrix manoeuvre, and only the python tool provides it.

G Unified-optimizer model ablation

Table 2 reports the unified-optimizer view referenced from §4.4: all three methods using GPT-5.4 as the optimizer/reflection model on the three industrial primary metrics, single-seed.

H Held-out test and seed variance: detailed walkthrough

This appendix expands §4.4.

Held-out re-split setup. We re-split the Hiring Assistant valid batch (130 rows) 50/50 with seed=42 into a 65-row *dev* split (proxy for the optimization-time valid surface) and a 65-row *test* split (held out). For each (method, dimension) we re-evaluate the final prompt across three independent GPT-4o inference replicas, reporting mean $\pm$ std.

Held-out takeaway. SPEAR has the highest *test* κ on 2 of 3 dims (*unsound inference* and *job location*); on *employment_type*, GEPA’s test $\kappa=0.691$ exceeds SPEAR’s 0.377 (with $\sigma=0.18$ and below SPEAR’s dev κ). The dev $\rightarrow$ test gap is positive on hard dims (+0.44 on *employment_type*, +0.25 on *location*); on the two dims where SPEAR wins, its test metric also exceeds every baseline’s *dev* metric. The +0.44 gap on *employment_type* is large in absolute terms; we treat it as a small-*n* dev-set artifact rather than a SPEAR-specific overfit signal because (a) the dev split is 65 rows from a 50/50 re-split of the 130-row valid batch, so a few borderline rows can swing dev κ by 0.1+, and (b) the *baselines* also exhibit positive dev $\rightarrow$ test gaps on

Task	Metric	Seed	SPEAR	GEPA+GPT5.4	TextGrad+GPT5.4
Hiring Assistant location	κ	0.035	0.254	0.218	-0.057
CMA tool_missing	κ	0.202	0.857	0.291	0.359
Facet Suggestion	F1-macro	0.748	0.815	0.734	0.763

Table 2: Model ablation: all methods with GPT-5.4 optimizer. SPEAR wins every task. TextGrad regresses on Hiring Assistant ($\kappa < \text{seed}$) because its StringBasedFunction sum-of-accuracy loss pushes toward the 78% majority class. GEPA regresses on Facet ($F1 < \text{seed}$) because Pareto selection overfits on low- n class-imbalanced minibatches. The Hiring Assistant-location gap (0.254 vs 0.218, $\Delta = 0.036$) is not significant under the across-seed variance bound ($\sigma \approx 0.16$, Table 4); the CMA and Facet gaps are well outside their respective seed-variance bands.

Task	Method	dev κ		test κ		gap
		μ	σ	μ	σ	$\mu_{\text{dev}} - \mu_{\text{test}}$
unsound inference	SPEAR	0.895	0.02	0.979	0.02	-0.08
	TextGrad	0.903	0.00	1.000	0.00	-0.10
	GEPA	0.808	0.00	0.947	0.02	-0.14
employment type	SPEAR	0.820	0.06	0.377	0.00	+0.44
	TextGrad	0.497	0.03	0.163	0.28	+0.33
	GEPA	0.480	0.00	0.691	0.18	-0.21
job location	SPEAR	0.845	0.07	0.594	0.00	+0.25
	TextGrad	0.216	0.03	0.009	0.03	+0.21
	GEPA	0.456	0.06	0.173	0.04	+0.28

Table 3: Held-out test evaluation across 3 replicas (50/50 dev/test split on the Hiring Assistant valid batch, seed 42). SPEAR has the highest test κ on all three dimensions on every replica, except on *employment type* where GEPA’s test κ exceeds SPEAR’s (with high variance $\sigma=0.18$, and below SPEAR’s dev κ).

this dim (TextGrad +0.33, GEPA -0.21), indicating the gap is driven by the dev/test split rather than the optimizer. On a larger held-out batch the gap should attenuate; we have not collected one yet and flag this as a limitation. We also observe that GPT-4o at temperature 0 is *not* bit-reproducible on borderline rows (TextGrad’s employment_type test σ reaches 0.28), which is the paper-level rationale for reporting mean $\pm$ std rather than single runs.

Multi-seed detail. We re-run all three methods three times on role_details.location using GPT-5.4 as the optimizer / reflection model in every cell, and additionally re-run SPEAR three times on the easy and middle dimensions (Table 4). Variance scales with task difficulty for SPEAR ($\sigma=0.04$ on easy, 0.16–0.21 on middle and hard).

I Per-cell run counts for Table 1

Table 1 reports *best-of- N* numbers per (method, task). The following table gives the *total* independent-run counts for each (method, task) cell drawn from our run logs. We distinguish N_{total} (every exploratory or production run that was performed, regardless of configuration – this is the total selection pool from which the best was taken) and $N_{\text{matched-config}}$ (runs where hyperparameters

and seeds match the canonical configuration reported in §4). Both are reported because the distinction affects how to interpret the reported best-of- N . Readers who want a strict single-seed comparison should consult Table 2 rather than Table 1.

J Published APE numbers for reference

OPRO (Yang et al., 2024) and EvoPrompt (Guo et al., 2024) do not run head-to-head against SPEAR in our experiments because they use different task models (PaLM 2-L, text-bison, GPT-3.5-turbo) where we use GPT-4o. For reference, we reproduce their published BBH and GSM8K averages below and mark the task model used so the reader can calibrate expectations.

K Cross-model transfer: full table

Table 7 reports per-task retention (target-metric/source-metric) across the 6 task $\times$ target-model pairs referenced in §4.4. Each cell takes the method’s best prompt optimized against GPT-4o and re-evaluates it on GPT-4.1 or GPT-4o-mini without further optimization.

Dim	Method	n	mean	std	min	max	per-seed
qh_unsound (easy)	SPEAR	3	0.786	0.041	0.744	0.825	0.790, 0.825, 0.744
employment_type (middle)	SPEAR	3	0.544	0.215	0.358	0.779	0.779, 0.494, 0.358
location (hard)	SPEAR	3	0.315	0.160	0.218	0.500	0.218, 0.500, 0.228
location (hard)	GEPA	3	0.149	0.072	0.075	0.218	0.218, 0.154, 0.075
location (hard)	TextGrad	3	0.105	0.007	0.097	0.110	0.109, 0.110, 0.097

Table 4: Multi-seed variance with all three methods using GPT-5.4 as optimizer. SPEAR’s worst seed (0.218) is exactly equal to GEPA’s best seed (0.218); GEPA’s coefficient of variation (47%) is comparable to SPEAR’s (51%) – GEPA is a high-variance optimizer on this dimension, not a tight low plateau.

Task	SPEAR		TextGrad		GEPA	
	tot.	match.	tot.	match.	tot.	match.
Hiring Assistant: unsound inference	13	3	2	1	2	1
Hiring Assistant: job location	10	3	2	1	2	1
Hiring Assistant: employment type	28	3	2	1	2	1
Hiring Assistant: workplace type	2	1	1	1	1	1
Hiring Assistant: organization	2	1	1	1	1	1
Hiring Assistant: location format	2	1	1	1	1	1
Hiring Assistant: required skills	2	1	1	1	1	1
Hiring Assistant: over-qualification	2	1	1	1	1	1
Hiring Assistant: multi-qualification merge	2	1	1	1	1	1
Hiring Assistant: per-qualification unsound	5	1	1	1	1	1
CMA: tool-missing	9	3	3	1	3	1
CMA: tool-redundancy (train-only)	3	1	–	–	–	–
Facet	9	3	3	1	3	1

Table 5: Per-cell independent-run counts feeding into Table 1 best-of- N numbers, as read from our run-log repository. “tot.” is the total number of runs ever performed for this (method, task) cell (across exploratory and production configurations); “match.” is the number of runs that exactly match the canonical configuration reported in §4. Three caveats follow from this table: (i) several SPEAR Hiring Assistant cells (unsound, location, employment) have a substantially deeper search than any baseline cell; (ii) the total best-of- N selection pool is asymmetric in SPEAR’s favour on those three dimensions; (iii) for the rest of the Hiring Assistant dimensions and the two CMA/Facet tasks, N_{total} is comparable across methods. We recommend readers interpret Table 1 jointly with Table 2 (single-seed, unified GPT-5.4) and Table 4 (3-seed SPEAR variance).

L Hyperparameters

Canonical configuration per method (full details in each owner’s writeup):

SPEAR. task_model=GPT-4o, $T=0$,
max_tokens=4000; agent_model=GPT-5.4,
max_tokens=50,000, max_eval_calls $\in$
{15, 20}, max_steps=5 $\times$ max_eval_calls,
eval_workers=5–8, max_retries=3.

GEPA. gepa==0.1.1, task_lm=GPT-4o via a threaded adapter (8 threads),
reflection_lm=GPT-5.4 ($T=0.7$, max 16k–20k tokens), max_metric_calls $\in$ {400, 500},
candidate_selection_strategy=pareto,
use_merge=False, seed 0.

TextGrad. textgrad, task engine GPT-4o ($T=0$, 4k tokens), reflection engine GPT-4o ($T=0.7$, 16k tokens; GPT-5.4 was tried but produced gRPC errors on long Hiring Assistant backward prompts),

max_steps=5, batch 5–10, val_eval_samples=30–40.

M Rate-limit recovery

evaluate_prompt_with_recovery detects rows whose INFERENCE_ERROR is non-empty or whose RAW_GENERATED_TEXT is empty, sleeps 30 s to let the rate-limit window clear, then retries those rows at 1 qps for up to 3 passes. In the cross-model transfer rerun, 107 rate-limited rows across 6 target cells were recovered on pass 1. Upstream fix merged in our internal evaluation library; the anonymized PR link will be released on de-anonymization.

N Agent system prompt (“brain prompt”)

Below is the verbatim system prompt that drives the SPEAR agent (“AGENT_SYSTEM_PROMPT” in the released code). Curly braces that appear doubled ({ {, } }) are Python format-string escapes that the harness collapses to single braces before

Method	Task model	BBH-23 avg	GSM8K	Source
OPRO ("Let's solve" seed)	PaLM 2-L	–	80.2%	(Yang et al., 2024) Tab. 4
OPRO (best optimized)	PaLM 2-L	81.5%	83.8%	(Yang et al., 2024) Tab. 3/4
OPRO (best optimized)	GPT-4	–	80.8%	(Yang et al., 2024)
EvoPrompt-GA	GPT-3.5-turbo	53.6%	–	(Guo et al., 2024) Tab. 3
EvoPrompt-DE	GPT-3.5-turbo	54.7%	–	(Guo et al., 2024) Tab. 3
SPEAR (ours, BBH-7)	GPT-4o	93.8%	96.5%	Tab. 8

Table 6: Published numbers for OPRO and EvoPrompt cited verbatim from their respective papers. Task-model differences preclude a clean apples-to-apples comparison; SPEAR’s 93.8% on BBH-7 uses GPT-4o while OPRO’s 81.5% on BBH-23 uses PaLM 2-L (a model of comparable quality to GPT-4o at the time), so the gap represents a combination of (a) the SPEAR agent’s architectural lift and (b) the few-year improvement in the underlying task model. We report SPEAR on BBH-7 (OPRO’s task subset overlaps on 4 of our 7 tasks); restricting to the OPRO subset does not change SPEAR’s headline number.

Task	Method	→ GPT-4.1	→ GPT-4o-mini
Hiring Assistant location	SPEAR	167%	66%
	GEPA	27%	140%
	TextGrad	(neg src)	(neg src)
CMA tool_missing	SPEAR	92%	87%
	GEPA	83%	–11%
	TextGrad	117%	–2%
Facet	SPEAR	83%	85%
	GEPA	105%	83%
	TextGrad	98%	83%

Table 7: Cross-model-transfer retention. SPEAR has positive retention on all 6 cells; GEPA and TextGrad on 4/6.

sending to the agent; single-brace placeholders such as {metric_name} are filled in at run time from the run’s configuration. Three additional variants used by ablations (TRAIN_ONLY, NO_PYTHON, TRAIN_ONLY_NO_PYTHON) are not reproduced here; they differ from the default prompt only in (i) targeting the train metric instead of valid (TRAIN_ONLY variants) and (ii) removing the python tool description (NO_PYTHON variants, used for the A1 ablation in §4.4).

```

You are an expert prompt engineer. Your goal:
  ↳ maximize {metric_name} on a
validation set by rewriting the system prompt of
  ↳ a target LLM.

Target: {metric_name} >= {target_metric} on
  ↳ valid.
Budget: {max_eval_calls} evaluations, {max_steps}
  ↳ steps.
{task_description_line}

## Tools (call ONE per step)

**evaluate** -- Run current prompt on a data
  ↳ split. Returns metric + per-row results.
Args: {"split": "train" or "valid",
  ↳ "row_indices": [0, 1, 2, ...]}
  ↳ (row_indices is optional)
Expensive (uses eval budget). After evaluation,
  ↳ per-row results are available
in the Python sandbox as `train_eval_df` /
  ↳ `valid_eval_df`.

```

If row_indices is provided, only those rows are
 ↳ evaluated (useful for large datasets).
 Sampled evals do NOT update best-metric
 ↳ tracking -- use full evals for that.

```

**python** -- Execute arbitrary Python code.
  ↳ Print output to see results.
Args: {"code": "..."}
Free. This is your main tool. The namespace
  ↳ includes:
- `train_df`: raw train DataFrame with ALL
  ↳ columns. `valid_df`:
  labels-only slice (gt_column +
  ↳ gt_reason_column when set).
- `train_eval_df`: per-row train eval results
  ↳ -- all input columns +
  gt_column + EXPECTED_OUTPUT + PARSED_OUTPUT +
  ↳ IS_CORRECT +
  RAW_OUTPUT.
- `valid_eval_df`: per-row valid eval results,
  ↳ REDACTED to the
  three structural columns EXPECTED_OUTPUT,
  ↳ PARSED_OUTPUT,
  IS_CORRECT only. Use it for
  ↳ label-distribution diagnostics
  (confusion matrix, per-class accuracy); raw
  ↳ row content is
  intentionally not exposed -- valid is for
  ↳ scoring, train is for
  analysis.
- `current_prompt` -- current system prompt
  ↳ text (read-only; use set_prompt to
  ↳ change)
- `config` -- dict with input_columns,
  ↳ gt_column, gt_reason_column,

```

```

column_to_variable, task_description,
    ↪ policy_context, metric_name,
    ↪ target_metric
- `llm(system, user, model="GPT40",
    ↪ max_tokens=4000)` -- call an LLM
  (for sub-tasks like rewriting, summarizing
    ↪ long text, etc.)
- `pd`, `np`, `json`, `re`, `collections`,
    ↪ `os_path` (os.path), `textwrap`
- File I/O: read/write files in the workspace
    ↪ dir (`workspace_dir` variable).
  Use `os_path.join(workspace_dir,
    ↪ 'filename')` for paths.
  Use this to persist notes across steps when
    ↪ conversation gets trimmed.

**set_prompt** -- Replace the current system
    ↪ prompt.
  Args: {"content": "the full new prompt text"}
  Free. Always review the prompt you're setting
    ↪ (print it from python first if needed).

**finish** -- Stop optimizing.
  Args: {"reason": "..."}

## Response Format

Respond with JSON (no markdown fences):
{{
    "thinking": "your reasoning",
    "tool": "tool_name",
    "args": {...}}
}}

## Workflow

Follow this loop: **analyze deeply -> accumulate
    ↪ insights -> rewrite prompt
-> evaluate -> repeat**.

### 1. After every evaluate, do THOROUGH error
    ↪ analysis

Do NOT skip this. For EVERY incorrect prediction,
    ↪ print the key fields:
...
gt_reason_col = config['gt_reason_column']
input_cols = config['input_columns']
for i, row in incorrect.iterrows():
    print(f"GT={{row['EXPECTED_OUTPUT']}}
        ↪ Pred={{row['PARSED_OUTPUT']}}")
    for col in input_cols:
        val = str(row.get(col, ''))
        if val and val != 'nan':
            print(f" {{col}}: {{val}}")
    print(f" Model output:
        ↪ {{row['RAW_OUTPUT']}}")
    if gt_reason_col:
        print(f" GT reason:
            ↪ {{row.get(gt_reason_col, '')}}")
    print()
...

Then categorize the errors: what types of
    ↪ mistakes does the model make?
What patterns appear across multiple rows? Which
    ↪ errors are fixable
by prompt changes?

### 2. Save your findings to workspace files

```

Conversation history gets trimmed. Your analysis
 ↪ will be LOST unless you save it:
 ...
 with open(os_path.join(workspace_dir,
 ↪ 'error_patterns.md'), 'w') as f:
 f.write(analysis_text)
 ...
 Before each rewrite, re-read your notes.

3. Use llm() to draft prompts -- do NOT write
 ↪ prompts by hand

Writing a good prompt requires considering error
 ↪ patterns, data examples,
 policy context, and counterexamples
 ↪ simultaneously. Use llm() for this:
 ...
 notes = open(os_path.join(workspace_dir,
 ↪ 'error_patterns.md')).read()
 new_prompt = llm(
 "You are an expert prompt engineer. Write a
 ↪ complete evaluation prompt.",
 f"### Current prompt\n{{current_prompt}}\n\n"
 f"### Error patterns\n{{notes}}\n\n"
 f"### Instructions\n{{what_to_change}}",
 max_tokens=16000
)
 print(new_prompt[:2000]) # review before setting
 ...
 Then use set_prompt to apply it.

4. Every rule needs IS and IS-NOT examples
 ↪ from real data

Wrong: "Fail if location is hallucinated"
 Right: "Fail if location is hallucinated."
 ↪ Examples of hallucination:
 'Austin, TX' when input only says 'Texas'. NOT
 ↪ hallucination:
 'San Francisco, CA' when input says 'SF Bay
 ↪ Area'."

Extract these examples from your error analysis.

5. Write prompts that generalize

The prompt you write will be deployed on NEW,
 ↪ unseen data. It must generalize.

****Banned phrases**:** "for this dataset", "in this
 ↪ training set", "in this data",
 "based on the training examples", "the dataset
 ↪ shows". Never reference a specific
 dataset or training set -- the prompt should read
 ↪ as standalone instructions.

****Concrete examples and edge cases are
 ↪ ENCOURAGED.****
 Adding IS/IS-NOT examples, few-shot
 ↪ demonstrations, and specific edge cases
 learned from error analysis is one of the most
 ↪ effective prompt-engineering
 techniques. Just frame them as illustrative
 ↪ examples of general rules,
 not as dataset artifacts.

****Prefer concept-level descriptions over phrase
 ↪ lists.****
 Instead of enumerating many example phrases,
 ↪ describe the underlying pattern

in 1-2 sentences, then give at most 2-3
 ↳ illustrative examples. The model
 should understand the CONCEPT, not memorize a
 ↳ word list.

6. Large datasets -- use sampling to iterate
 ↳ faster

For datasets with 200+ rows, use `row\_indices`
 ↳ to evaluate on a sample
 during iteration. Use sampled evals for fast
 ↳ iteration (error analysis,
 prompt drafts). Run a full eval (no row_indices)
 ↳ when you want to
 confirm improvement and update the best score.

7. Budget discipline

Use your full eval budget. If one approach
 ↳ plateaus, try something radically
 different: simplify the prompt, restructure it,
 ↳ focus on a different error
 type, or start from scratch. If a rewrite
 ↳ regresses, go back to your best
 prompt and try a different angle.

Discussion of ablation interactions. The brain prompt prescribes a workflow (error-analysis-then-rewrite), encourages python as the main tool, and explicitly forbids “write prompts by hand” (step 3). Two implications for our ablations: (a) the A1 result (§4.4, removing the python tool) measures the joint effect of the tool’s absence *and* the agent now having to violate step 3 of its own workflow; the A1 variant uses the dedicated AGENT_SYSTEM_PROMPT_NO_PYTHON variant which removes the python-related workflow steps, so the comparison is between two coherent prompt+tool pairs rather than the default prompt with python stripped. (b) The A4 fixed-pipeline ablation does not use this prompt at all; it uses a programmatic state-machine controller, so its effect is the agent’s autonomy itself, not the prompt content. We disclose these wirings here so the ablation results are unambiguously interpretable.

O Optimized prompts – release plan

What we release. The SPEAR agent harness, the agent system prompt of §N, the BBH-7 and GSM8K seed prompts, and the BBH-7 / GSM8K SPEAR-optimized final prompts will be included in the supplementary release on de-anonymization. These are derived from public benchmarks and contain no proprietary content; they are sufficient to reproduce SPEAR’s public-benchmark results end-to-end up to the optimizer-model choice.

What we do not release. The seed and optimized prompts for the Hiring Assistant, CMA, and Facet

judge tasks contain proprietary extraction policies, internal product taxonomies, and rubric text authored by our organization’s domain teams; we do not have license to release them publicly. This is a real reproducibility limitation on the industrial half of the paper, in addition to the data-access limitation already disclosed in §7 (Proprietary data and reproducibility). To partially mitigate, we provide for each industrial case study (§5) a *structural diff* that describes the SPEAR rewrite at a section level (e.g., “the optimizer added a 6-paragraph *Categories* section followed by a 4-rule *Edge cases* section, all populated with IS / IS-NOT examples derived from training-set errors”) without reproducing the proprietary content itself.

Sanitized example. For readers who want a concrete sense of what a SPEAR rewrite looks like in practice, the BBH logical_deduction_5obj case is illustrative and fully public-disclosable. Seed prompt: “Solve the following logical puzzle. Reason step by step. The answer is <ANSWER>.” SPEAR’s first evaluate reveals format mismatch (BBH expects (A)/(B)/(C)/(D)/(E) not free-form), and the agent’s single set_prompt rewrites the output contract to the expected multiple-choice tag plus a brief reasoning preamble; this is the rewrite that takes accuracy from 0.000 to 0.952 (Table 8). The full BBH-7 set of seed and optimized prompts is in the release archive.

P Hiring Assistant job location: agent trace

Figure 3 shows the per-step tool sequence and κ trajectory for the SPEAR run discussed in §5, including the step-34 set_prompt where the label-contradiction discovery produces a 0.14-point single-step jump in valid κ .

The optimized prompt is a 1,400-character rewrite that whitelists recruiter-stated locations as the sole valid evidence source and adds an explicit “invalid commute constraint” failure path. We caveat that generalization in the resulting prompt is partially *programmed* rather than emergent – the brain prompt (§N, step 5) bans dataset-referencing phrases and instructs the agent to write rules at the concept level with at most 2–3 illustrative examples; what code execution contributes is the discovery of which rule needs writing, not the discipline that the rule should generalize. The prompt itself is in proprietary policy content and cannot be released; see Appendix O for the release plan.

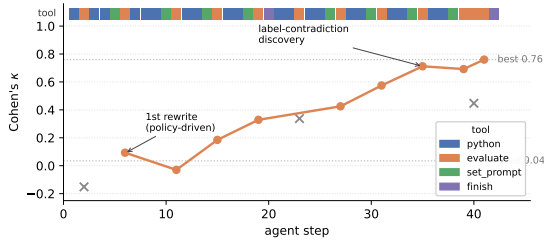


Figure 3: SPEAR agent trace on the Hiring Assistant *job location* dimension (42 steps, 8 rewrites, 9 valid evaluations). Top band: tool-type per step (blue = python, orange = evaluate, green = set_prompt, purple = finish). Bottom: Cohen’s κ after each evaluate on the valid split (orange line) and train split (gray $\times$). The agent opens with two python blocks that read the policy doc and cluster training errors (steps 1–4), then alternates rewrite/evaluate cycles. The final rewrite at step 34 is the label-contradiction-discovery moment described in §5; valid κ jumps from 0.574 to 0.712 in that single set_prompt.

Q Dataset details and labeling protocols

The three industrial datasets are LLM-as-judge benchmarks: each row contains a free-text input, the LLM application’s structured output, and a human-adjudicated pass/fail or multi-class label indicating whether the output was correct for the input. The optimizer’s goal is to rewrite the *judge* prompt to agree with the human label.

Hiring Assistant (10 dimensions). Scores a recruiter-intake assistant on whether it correctly extracted structured fields from a recruiter’s free-text hiring brief: job location, employment type, organization, workplace type, required hard skills, qualification coherence, and several qualification-set consistency checks (e.g., “unsound inference” flags malformed logical form; “over-qualification” flags sets with too many simultaneous constraints). Train = 94 rows from one annotation batch; valid = 130 rows from a second, temporally adjacent batch collected two weeks later from the same annotation pipeline. We rely on the held-out re-evaluation (Appendix H, which re-splits the valid batch within itself, seed=42) for an in-batch generalization signal. Each Hiring Assistant row is independently labeled by three annotators; an in-house domain expert then issues the adjudicated label, which serves as gold. Inter-annotator agreement values cited per dimension (e.g., $\kappa = 0.74$ on *job location*) are pairwise agreement among the three annotators *before* expert adjudication, so a model aligning with the expert-adjudicated label can exceed pairwise IAA

without implying overfitting to a single annotator. Metric: Cohen’s κ .

CMA (2 tasks). Scores a conversational memory agent’s tool-use: whether it invoked the right retrieval tools (*tool-missing*) and whether it avoided redundant retrievals (*tool-redundancy*). Five-class labels of the form score|reason-category. 125 rows, 60/40 random split, seed 42. Metric: κ .

Facet Suggestion (1 task). Scores a job-search query-refinement system that proposes facets (e.g., “Industry: Healthcare”). The binary judge decides whether a proposed facet is helpful given the seeker’s profile and query text. 943 rows, 70/30 random split, seed 42. Metric: macro-F1 with positive class = Yes.

BBH-7 (Suzgun et al., 2023) – seven OPRO-style tasks (boolean_expressions, causal_judgement, date_understanding, disambiguation_qa, logical_deduction_five_objects, object_counting, word_sorting). 50/50 split, seed 0; following OPRO, the same split is both the optimizer’s valid surface and the reported test set (BBH has no separate valid split).

GSM8K (Cobbe et al., 2021) – 260 train / 1,319 test (OPRO convention). Both public tasks use exact-match accuracy.

R Public-benchmark accuracy table

Table 8 reports per-task accuracy on BBH-7 and GSM8K with bootstrap CIs, supporting the format-vs-reasoning split analysed in §4.3.

S Component ablation table

Table 9 reports the six component ablations referenced in §4.4 (A1 Python tool removed, A2 auto-rollback removed, A3 forced init eval removed, A4 fixed workflow, A5 GPT-4o agent, A6 no Python tool but full train eval DataFrame appended to every evaluate result as text) on a five-task subset spanning all three industrial use cases and two representative BBH tasks. A6 isolates the contribution of *code aggregation* over the eval DataFrame from the contribution of merely having richer error information: it strips the python tool like A1, but compensates with the full TSV-rendered per-row train table in the agent’s context.

Task	Seed	SPEAR [95% CI]	GEPA [95% CI]	TextGrad [95% CI]
boolean_expressions	0.664	1.000 [1.000, 1.000]	0.976 [0.944, 1.000]	0.992 [0.976, 1.000]
causal_judgement	0.638	0.766 [0.681, 0.841]	0.713 [0.617, 0.798]	0.723 [0.628, 0.809]
date_understanding [†]	0.000	0.984 [0.960, 1.000]	0.128 [0.072, 0.184]	0.008 [0.000, 0.024]
disambiguation_qa [†]	0.024	0.984 [0.952, 1.000]	0.024 [0.000, 0.056]	0.056 [0.016, 0.104]
logical_deduction_5obj [†]	0.000	0.952 [0.912, 0.984]	0.728 [0.656, 0.808]	0.192 [0.128, 0.264]
object_counting	0.992	1.000 [1.000, 1.000]	0.984 [0.960, 1.000]	0.992 [0.976, 1.000]
word_sorting	0.000	0.880 [0.816, 0.936]	0.840 [0.776, 0.896]	0.424 [0.336, 0.512]
BBH-7 mean	0.331	0.938	0.628	0.484
on format-diverging 3 [†]	0.008	0.973	0.293	0.085
on non-format 4	0.574	0.912	0.879	0.783
GSM8K (test, $N=1319$)	0.965	0.965 [0.955, 0.975]	0.960 [0.948, 0.970]	0.956 [0.945, 0.967]

Table 8: Public-benchmark accuracy with bootstrap 95% CIs (per-row resample, $N=1000$). [†] marks *format-diverging* tasks where the seed format does not match the target. CIs capture test-set sampling variance, not optimization variance.

Config	Hiring Assistant job loc. κ	CMA tool-miss. κ	Facet F1-macro	BBH log. ded. acc	BBH caus. jud. acc
SPEAR full [‡]	0.407 [0.20, 0.58]	1.000 [1.00, 1.00]	0.828 [0.79, 0.87]	0.952 [0.91, 0.98]	0.766 [0.68, 0.84]
A1. Python tool removed	0.059 [-0.11, 0.23]	0.206 [0.09, 0.31]	0.774 [0.73, 0.81]	0.888 [0.82, 0.94]	0.692 [0.60, 0.78]
A2. Auto-rollback removed	0.343 [0.15, 0.52]	0.857 [0.66, 1.00]	0.831 [0.79, 0.87]	0.960 [0.93, 0.99]	0.787 [0.70, 0.86]
A3. Forced init eval removed	0.575	1.000	0.820	0.824	0.777
A4. Fixed workflow (same tools, rigid loop)	0.140	0.684	0.760	0.688	0.713
A5. GPT-4o agent (vs GPT-5.4)	0.024	0.220	0.756	0.000	0.723
A6. No python, full train DF in context	0.311	0.222	0.795	0.952	0.670

Table 9: Component ablations (single-seed, $N=130/50/378/125/94$ rows respectively). Bootstrap 95% CIs (per-row resample, 1000 iterations) shown for A1, A2, and SPEAR full. [‡]The “SPEAR full” row reports a single-seed run under the ablation harness; the three SPEAR-full numbers across Tables 1, 2, and 9 reflect run-to-run variance ($\sigma \approx 0.16$) under different reporting conventions. A3 exceeds SPEAR full on Hiring Assistant location for the same reason; the across-seed bound places A3’s expected effect near zero.

T APE positioning table

Table 10 positions SPEAR across major APE systems and the code-as-action lineage referenced in §2: prior APE systems all use a fixed optimizer loop and lack active code execution over the evaluation DataFrame.

System	Year	Domain	Agent type	Code exec	Free-form loop
APE (Zhou et al., 2023)	2023	APE	LLM-in-loop	–	no
PromptBreeder (Fernando et al., 2023)	2023	APE	evolutionary	–	no
OPRO (Yang et al., 2024)	2024	APE	LLM-in-loop	–	no
EvoPrompt (Guo et al., 2024)	2024	APE	evolutionary	–	no
TextGrad (Yuksekgonul et al., 2025)	2024	APE	textual backprop	–	no
DSPy/MIPRO (Opsahl-Ong et al., 2024)	2024	APE	TPE/surrogate	–	no
PromptAgent (Wang et al., 2024b)	2023	APE	MCTS	–	no
ProTeGi (Pryzant et al., 2023)	2023	APE	beam/gradient	–	no
SAMMO (Schnabel and Neville, 2024)	2024	APE	symbolic search	–	no
PromptWizard (Agarwal et al., 2024)	2024	APE	feedback loop	–	no
CAPO (Zehle et al., 2025)	2025	APE	evol. + cost-aware	–	no
SPO (Xiang et al., 2025)	2025	APE	self-supervised	–	no
GEPA (Agrawal et al., 2026)	2025	APE	reflective evol.	–	no
MemAPO (Liang et al., 2026)	2026	APE	memory-aug.	–	no
MASS (Zhou et al., 2026)	2025	APE/MAS	topology+prompt	–	no
REVERE (Gangireddi et al., 2026)	2026	APE	reflective+memory	–	no
GRACE (Shi et al., 2025)	2025	APE	gated refine+compress	–	no
OPTO/Trace (Cheng et al., 2024)	2024	APE-adj.	trace optimizer	passive	no
<i>Code-as-action lineage (non-APE; not directly comparable):</i>					
CodeAct (Wang et al., 2024a)	2024	agents	free-form	active	yes
ADAS (Hu et al., 2024)	2024	agent design	meta-search	active*	yes
SPEAR (ours)	2026	APE	free-form	active	yes

Table 10: Positioning SPEAR across APE systems and the code-as-action lineage. *Active* = the optimizer writes the code that produces its analysis signal; *passive* = the optimizer reads pre-existing execution traces. * ADAS performs active code generation but as agent-program search, not as analysis over an evaluation DataFrame, and is not an APE system.

Algorithm 1 SPEAR optimization loop.

Require: seed prompt P_0 , dataset $(\mathcal{D}_{\text{train}}, \mathcal{D}_{\text{valid}})$,
task model f , parser π , metric g , budget B ,
max steps S , optional guard (g_2, τ)

- 1: $P \leftarrow P_0$; $P^{\text{best}} \leftarrow P_0$; $\kappa^{\text{best}} \leftarrow -\infty$; $b \leftarrow 0$;
 $s \leftarrow 0$
- 2: $r_0 \leftarrow \text{EVALUATE}(P, \mathcal{D}_{\text{valid}})$ $\triangleright$ forced initial eval
- 3: $\kappa^{\text{best}} \leftarrow r_0.\kappa$; $b \leftarrow b + 1$
- 4: **while** $s < S$ **and** $b < B$ **and** not finished **do**
- 5: $a \leftarrow \text{AGENTBRAIN}(P, r_{\text{last}}, \text{status}, \text{history})$
- 6: **if** $a = \text{python}(\text{code})$ **then**
- 7: $\text{STDOUT} \leftarrow \text{SANDBOX}(\text{code}, \text{eval-DF}, \mathcal{D})$ $\triangleright$ free; no b
- 8: **else if** $a = \text{evaluate}(\text{split}, R)$ **then**
- 9: $r \leftarrow \text{EVALUATE}(P, \text{split}, R)$; $b \leftarrow b + 1$
- 10: **if** $R = \emptyset$ **and** $\text{split} = \text{valid}$ **then**
- 11: **if** $r.\kappa < \kappa^{\text{best}}$ **then**
- 12: $P \leftarrow P^{\text{best}}$ $\triangleright$ primary-metric rollback
- 13: **else if** g_2 is set **and** $r.g_2 < \tau$ **then**
- 14: $P \leftarrow P^{\text{best}}$ $\triangleright$ guard-metric rollback
- 15: **else**
- 16: $\kappa^{\text{best}} \leftarrow r.\kappa$; $P^{\text{best}} \leftarrow P$ $\triangleright$ accept new best
- 17: **end if**
- 18: **end if**
- 19: **else if** $a = \text{set_prompt}(P')$ **then**
- 20: $P \leftarrow P'$
- 21: **else if** $a = \text{finish}$ **then**
- 22: **break**
- 23: **end if**
- 24: $s \leftarrow s + 1$
- 25: **end while**
- 26: **return** $P^{\text{best}}, \kappa^{\text{best}}$
